Langage C — fondamentaux

Informatique — Licence 1 (L1)

Cours INF-L1-02

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Découvrir la programmation en langage C, de l'installation de l'environnement aux structures de données composées : variables, entrées/sorties, conditions, boucles, fonctions, tableaux et chaînes.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Installation de l'environnement et premier programme en C
• Leçon 2 — Variables, constantes et opérateurs
• Leçon 3 — Entrées et sorties : printf et scanf
• Leçon 4 — Conditions et boucles en C
• Leçon 5 — Fonctions : déclaration, appel et portée des variables
• Leçon 6 — Tableaux et chaînes de caractères
• Leçon 7 — Structures et types composés
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Le langage C, créé au début des années 1970, reste le langage de référence pour comprendre comment un ordinateur fonctionne réellement : c'est en C qu'ont été écrits les systèmes d'exploitation, les pilotes et une grande partie des logiciels qui font tourner le monde. Apprendre le C en première année n'est donc pas un détour historique : c'est la manière la plus directe d'acquérir des bases solides — types stricts, mémoire visible, compilation — qui rendront ensuite Python, Java ou JavaScript beaucoup plus faciles à comprendre.

Ce cours fait suite à l'algorithmique : chaque notion de pseudo-code que tu connais déjà (variable, condition, boucle, fonction) trouve ici sa traduction exacte en C. La différence majeure est la rigueur : le C exige des points-virgules, des types déclarés et des adresses mémoire explicites pour lire une variable au clavier. Cette rigueur, parfois frustrante au début, forme des programmeurs précis.

À la fin du cours, tu sauras écrire, compiler et exécuter des programmes complets qui demandent des données à l'utilisateur, calculent, décident et manipulent des tableaux de notes ou des fiches d'étudiants — exactement le niveau attendu en fin de L1 pour aborder le cours avancé sur les pointeurs et les fichiers.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Installer un environnement de développement C et expliquer les étapes édition → compilation → exécution.
- Déclarer variables et constantes avec les types de base (int, float, double, char) et utiliser les opérateurs.
- Lire et afficher des données avec scanf et printf, avec les bons spécificateurs de format.
- Écrire des conditions (if, else, switch) et des boucles (for, while, do-while) correctes.
- Découper un programme en fonctions avec paramètres et valeur de retour, en maîtrisant la portée des variables.
- Manipuler tableaux et chaînes de caractères, y compris les fonctions de <string.h>.
- Regrouper des données liées dans une structure (struct) et l'utiliser dans un petit programme complet.

Leçon 1 — Installation de l'environnement et premier programme en C

Un programme C commence sa vie comme un simple fichier texte, portant l'extension .c. Pour le transformer en programme exécutable, il faut un compilateur — le plus répandu est GCC, gratuit — souvent intégré à un environnement de développement (IDE) comme Code::Blocks ou Visual Studio Code, qui ajoute un éditeur coloré et un bouton « exécuter ». Sous Windows, on installe généralement Code::Blocks avec MinGW inclus ; sous Linux, GCC est souvent déjà là ; dans tous les cas, un premier test s'impose avant d'écrire la moindre ligne sérieuse.

La chaîne de production comporte trois étapes. L'édition : tu écris le code source. La compilation : le compilateur traduit le source en langage machine et signale les erreurs de syntaxe (point-virgule oublié, accolade manquante) — c'est à cette étape que se joue la plupart des batailles du débutant. L'exécution : le programme produit enfin ses affichages. Comprendre cette séparation explique un mystère fréquent : un programme peut compiler parfaitement et pourtant donner des résultats faux — la compilation vérifie la grammaire, pas la justesse du raisonnement.

Tout programme C possède une fonction main, point d'entrée de l'exécution, et commence presque toujours par inclure la bibliothèque standard d'entrées/sorties, <stdio.h>, qui fournit printf et scanf. L'instruction return 0, en fin de main, signale au système que tout s'est bien passé. Le caractère \n dans une chaîne demande un retour à la ligne à l'affichage.

L'essentiel
Édition du .c → compilation par GCC (qui vérifie la syntaxe) → exécution. main est le point d'entrée ; #include <stdio.h> donne accès à printf et scanf.

Exemple — Le premier programme

```
#include <stdio.h>

int main(void)
{
printf("Bonjour, bienvenue a l'IAM !\n");
printf("Mon premier programme C.\n");
return 0;
}
```

À retenir

• Un programme C est un fichier texte compilé en langage machine avant d'être exécuté.

• Les erreurs de compilation (syntaxe) se distinguent des erreurs de logique (résultat faux).

• Chaque instruction se termine par un point-virgule ; les blocs sont délimités par des accolades { }.

Exercice

Écris un programme C qui affiche trois lignes : ton prénom, ta filière et l'année universitaire, chacune sur sa propre ligne. Puis explique ce qui se passe, étape par étape, si tu retires le \n de la première ligne d'affichage.

✔ Corrigé
Trois appels à printf, chacun terminé par \n, produisent trois lignes distinctes. Sans le premier \n, le texte suivant se colle immédiatement après le prénom, sur la même ligne : \n n'ajoute pas une ligne automatiquement en fin de programme, il déplace le curseur au moment où il est affiché. Le programme compile dans les deux cas — preuve qu'une sortie « moche » n'est pas une erreur de syntaxe mais un comportement à prévoir.

Leçon 2 — Variables, constantes et opérateurs

En C, toute variable doit être déclarée avec son type avant usage : int pour les entiers, float et double pour les nombres à virgule (double étant le plus précis et le plus utilisé), char pour un caractère unique. La déclaration réserve un emplacement mémoire dont la taille dépend du type : un int occupe typiquement 4 octets. Une variable déclarée mais non initialisée contient n'importe quoi — la valeur qui traînait en mémoire ; la règle d'or est donc d'initialiser dès la déclaration chaque fois que c'est possible.

Les constantes nomment des valeurs qui ne changeront jamais : soit avec le mot-clé const (const float TVA = 0.18;), soit avec une directive #define qui remplace textuellement le nom par la valeur avant compilation. Nommer ses constantes rend le programme lisible et les changements faciles : si la TVA passe à 20 %, on modifie un seul endroit.

Les opérateurs de base sont + − * / et % (modulo, reste de la division entière). Deux pièges dominent la pratique. D'abord, la division entre entiers est entière : 7 / 2 vaut 3 en C, pas 3,5 ; pour obtenir un résultat réel, au moins un des deux nombres doit être

réel (7.0 / 2 vaut 3,5). Ensuite, les priorités sont celles des mathématiques : la multiplication passe avant l'addition, et les parenthèses tranchent en cas de doute. Les opérateurs composés (+=, −=, *=) et l'incrémentation (++) abrègent les mises à jour : compteur++ ajoute 1 à compteur.

L'essentiel
Déclarer = donner un type et un nom avant usage, et initialiser tout de suite. Division entière entre entiers (7/2 = 3) ; % donne le reste. const et #define pour les valeurs fixes.

Exemple — Calcul d'une moyenne avec les bons types

```
#include <stdio.h>

int main(void)
{
int note1 = 12, note2 = 15, note3 = 9;
double moyenne;

moyenne = (note1 + note2 + note3) / 3.0;
printf("Moyenne = %.2f\n", moyenne);
return 0;
}
// Affiche : Moyenne = 12.00
// Avec / 3 au lieu de / 3.0, on obtiendrait 12 aussi ici,
// mais (12+15+10)/3 donnerait 12 au lieu de 12.33 !
```

À retenir

- int = entiers, float/double = réels, char = un caractère ; double est le choix par défaut pour les réels.
- 7 / 2 = 3 (division entière) mais 7.0 / 2 = 3.5 : le type des opérandes décide du résultat.
- Une variable non initialisée contient une valeur imprévisible : toujours initialiser.

Exercice

Sans l'exécuter, donne ce qu'affiche ce fragment : int a = 17, b = 5; printf("%d %d %.1f", a / b, a % b, a / (double)b); — puis explique le rôle de la conversion (double) dans le troisième affichage.

✔ Corrigé
a / b = 17 / 5 = 3 (division entière, on garde le quotient). a % b = 2 (le reste : 17 = 5 × 3 + 2). a / (double)b convertit b en réel avant la division : le calcul devient réel, 17 / 5,0 = 3,4, affiché « 3.4 » grâce au format %.1f. La conversion explicite, appelée cast, est la manière propre de forcer une division réelle quand les deux variables sont entières.

Leçon 3 — Entrées et sorties : printf et scanf

printf affiche du texte et des valeurs. Les valeurs s'insèrent dans la chaîne grâce à des spécificateurs de format : %d pour un entier, %f pour un réel, %c pour un caractère, %s pour une chaîne. On peut préciser l'affichage : %.2f limite un réel à deux décimales (indispensable pour les montants), %5d réserve cinq caractères de large. Le nombre et l'ordre des spécificateurs doivent correspondre exactement aux valeurs fournies après la chaîne — un décalage produit des affichages absurdes sans erreur de compilation.

scanf fait le travail inverse : il lit ce que l'utilisateur tape au clavier et le range dans des variables. Sa particularité, qui déroute tous les débutants, est qu'il lui faut l'adresse des variables : scanf("%d", &age) — le symbole & signifie « adresse de ». Oublier le & est l'erreur la plus célèbre du langage C : le programme compile parfois, puis plante ou lit n'importe quoi. scanf renvoie d'ailleurs le nombre de valeurs correctement lues, ce qui permet de vérifier la saisie dans les programmes soignés.

Pour lire une chaîne sans espaces, on utilise %s avec le nom du tableau de caractères (sans &, car un tableau est déjà une adresse). Pour lire une ligne entière avec des espaces, fgets est l'outil adapté — retenir simplement, à ce niveau, que %s s'arrête au premier espace. Un dernier réflexe : toujours afficher une consigne claire avant une lecture (« Entrez votre âge : »), sinon l'utilisateur fixe un curseur qui clignote sans comprendre ce qu'on attend de lui.

L'essentiel
printf : %d, %f (%.2f pour deux décimales), %c, %s. scanf exige l'adresse : &variable pour les types simples. Pour une chaîne, %s sur le nom du tableau, sans &.

Exemple — Dialogue complet avec l'utilisateur

```
#include <stdio.h>

int main(void)
{
char nom[30];
int age;
double taille;

printf("Votre nom : ");
scanf("%s", nom);
printf("Votre age : ");
scanf("%d", &age);
printf("Votre taille (en m) : ");
scanf("%lf", &taille);

printf("%s, %d ans, %.2f m\n", nom, age, taille);
return 0;
}
```

À retenir

- Le format doit correspondre au type : %d ↔ int, %lf ↔ double dans scanf, %f suffit dans printf.

• Le & devant la variable dans scanf n'est pas décoratif : c'est l'adresse où ranger la valeur.

• Toujours guider l'utilisateur par un message avant chaque saisie.

Exercice

Écris un programme qui demande le prix unitaire d'un article et une quantité, puis affiche le total à payer avec deux décimales. Indique précisément les déclarations, les formats utilisés dans scanf et printf, et ce qui se passerait si l'on oubliait le & devant la quantité.

✔ Corrigé
Déclarations : double prix; int quantite; double total. Lectures : scanf("%lf", &prix) et scanf("%d", &quantite). Calcul : total = prix * quantite. Affichage : printf("Total : %.2f FCFA\n", total). Sans le & sur quantite, scanf reçoit la valeur contenue dans la variable (au lieu de son adresse) et tente d'écrire la saisie à cette « adresse » arbitraire : comportement indéfini, en pratique un plantage du programme — l'erreur ne se voit pas à la compilation, seulement à l'exécution, ce qui la rend redoutable.

Leçon 4 — Conditions et boucles en C

Le if du C reprend exactement la condition de l'algorithmique : if (condition) { … } else { … }, et les cascades else if enchaînent les cas. Deux différences avec le pseudo-code : la condition est entre parenthèses obligatoires, et l'égalité se teste avec == alors que = affecte — écrire if (x = 5) compile, attribue 5 à x et vaut toujours « vrai » : c'est LE piège historique du C. Les opérateurs logiques s'écrivent && (ET), || (OU) et ! (NON). Le switch, lui, compare une variable entière ou un caractère à une liste de cas, chacun terminé par break, avec un cas default pour le reste.

Les trois boucles du cours d'algorithmique existent : for (init; condition; mise à jour) pour les parcours comptés, while (condition) pour répéter tant qu'une condition tient — testée avant chaque tour, donc zéro tour possible — et do { … } while (condition), qui exécute au moins une fois et sert aux validations de saisie. break sort immédiatement de la boucle ; continue saute au tour suivant. Une boucle dont la condition ne change jamais est infinie : le programme semble « figé » et il faut l'interrompre.

Le réflexe de vérification, avant même d'exécuter : simuler deux ou trois tours à la main en suivant la variable qui contrôle la boucle. Neuf boucles fausses sur dix le sont par la borne (i < n au lieu de i <= n selon que l'on compte de 0 ou de 1) ou par une mise à jour oubliée dans un while. En C, les tableaux et les boucles comptent presque toujours à partir de 0 — c'est la convention à adopter dès maintenant.

L'essentiel
== compare, = affecte : ne jamais confondre dans un if. for = compté, while = condition d'abord, do-while = au moins un tour. break sort, continue saute un tour.

Exemple — Compter les notes au-dessus de la moyenne

```
#include <stdio.h>

int main(void)
{
int note, compteur = 0, i;

for (i = 1; i <= 5; i++) {
printf("Note %d : ", i);
scanf("%d", &note);
if (note >= 10) {
compteur++;
}
}
printf("%d note(s) au-dessus de 10\n", compteur);
return 0;
}
```

À retenir

- if (x = 5) affecte au lieu de comparer : toujours écrire == dans un test.
- La borne d'une boucle se vérifie en simulant le premier et le dernier tour.
- do-while valide une saisie naturellement : on demande, puis on recommence si la valeur est hors limites.

Exercice

1) Écris un programme qui demande un nombre entier n et affiche la somme des entiers de 1 à n à l'aide d'un while. 2) Le programme suivant contient une erreur — trouve-la et corrige-la : int i = 0; while (i < 10) { printf("%d ", i); }

✔ Corrigé
1) int n, i = 1, somme = 0; lire n; while (i <= n) { somme += i; i++; } afficher somme. 2) La boucle est infinie : i ne change jamais dans le corps, donc la condition i < 10 reste éternellement vraie et le programme affiche des 0 sans fin. Correction : ajouter i++ dans le corps de la boucle (ou transformer en for). Toute boucle while doit modifier, à chaque tour, au moins une variable de sa condition.

Leçon 5 — Fonctions : déclaration, appel et portée des variables

Une fonction C regroupe des instructions sous un nom, avec un type de retour, des paramètres typés et un corps entre accolades. Pour l'utiliser avant de la définir, on écrit d'abord son prototype — la signature terminée par un point-virgule — qui annonce au compilateur le nom, les paramètres et le type de retour ; la définition complète peut venir après main. Une fonction qui ne rend rien a le type void : c'est l'équivalent d'une procédure. L'appel remplace les paramètres formels par les valeurs effectives, exécute le corps, et return rend la valeur à l'appelant.

En C, les paramètres sont transmis par valeur : la fonction reçoit une copie des arguments. Modifier le paramètre dans la fonction ne modifie donc pas la variable

d'origine chez l'appelant — pour changer une variable de l'extérieur, il faudra les pointeurs (cours INF-L1-03). Cette règle, claire et sans exception pour les types simples, évite des effets de bord invisibles : une fonction ne peut abîmer que ce qu'on lui a explicitement confié… en copie.

La portée règle la visibilité des variables : une variable déclarée dans une fonction est locale — elle naît à l'entrée, meurt à la sortie, et deux fonctions peuvent utiliser le même nom sans interférer. Une variable déclarée hors de toute fonction est globale, visible partout : pratique en apparence, dangereuse en réalité, car n'importe quelle fonction peut la modifier et les erreurs deviennent intraçables. Le style professionnel minimise les globales et fait circuler les données par les paramètres et les retours.

L'essentiel
Prototype = la signature annoncée ; définition = le code ; appel = l'exécution avec des valeurs. Passage par valeur (copie) pour les types simples. Locales par défaut, globales à éviter.

Exemple — Une fonction de calcul et son utilisation

```
#include <stdio.h>

double prixTTC(double prixHT); // prototype

int main(void)
{
double p = prixTTC(10000.0);
printf("Prix TTC : %.2f FCFA\n", p);
return 0;
}

double prixTTC(double prixHT) // definition
{
return prixHT * 1.18;
}
```

À retenir

- Sans prototype ni définition avant l'appel, le compilateur refuse la fonction inconnue.
- Le paramètre est une copie : la fonction ne modifie pas la variable de l'appelant (types simples).
- Une variable locale n'existe que pendant l'exécution de sa fonction.

Exercice

Écris la fonction int maximum(int a, int b) qui retourne le plus grand des deux nombres, puis un main qui lit deux entiers au clavier et affiche le maximum en l'utilisant. Explique ensuite ce qui se passe si l'on modifie a à l'intérieur de maximum : la variable de main change-t-elle ?

✔ Corrigé
int maximum(int a, int b) { if (a >= b) return a; else return b; } — main lit x et y, appelle maximum(x, y) et affiche le résultat. Si maximum modifie son paramètre a, cela ne change que sa copie locale : la variable x de main garde sa valeur. Pour modifier réellement la variable de l'appelant, il faudra passer son adresse avec un pointeur — c'est précisément l'objet du prochain cours.

Leçon 6 — Tableaux et chaînes de caractères

Un tableau C se déclare avec sa taille entre crochets : int notes[30]; et ses cases sont numérotées de 0 à 29 — en C, on commence toujours à zéro, et l'indice taille−1 est le dernier valide. Demander notes[30] est un débordement : le programme lit ou écrit chez le voisin, sans aucune garantie. On parcourt le tableau par une boucle for de 0 à taille−1, et lorsqu'on passe un tableau à une fonction, on passe en réalité son adresse de début : la fonction travaille donc sur l'original, et il faut lui fournir aussi la taille, qu'elle ne peut pas deviner.

Une chaîne de caractères est un tableau de char terminé par un caractère spécial, '\0', qui marque la fin : le mot « IAM » occupe 4 cases — I, A, M, \0. Cette sentinelle est indispensable : sans elle, les fonctions ne savent pas où la chaîne s'arrête. On déclare char nom[30] pour une chaîne de 29 caractères utiles au maximum, on la lit par scanf("%29s", nom) — la limite protège des débordements — et on l'affiche par %s.

La bibliothèque <string.h> fournit les outils des chaînes : strlen(nom) donne la longueur sans le \0, strcpy(destination, source) copie, strcmp(a, b) compare (0 si égales, négatif ou positif selon l'ordre alphabétique), strcat colle deux chaînes. Le piège à retenir : on ne peut ni affecter (nom = "Awa" est interdit après la déclaration — il faut strcpy) ni comparer (nom == "Awa" compare des adresses, pas des contenus — il faut strcmp). Ces deux erreurs sont les plus fréquentes de L1 en C.

L'essentiel
Indices de 0 à taille−1 ; tout dépassement est un débordement dangereux. Une chaîne = un tableau de char terminé par '\0'. Copier avec strcpy, comparer avec strcmp, mesurer avec strlen.

Exemple — Remplir un tableau de notes et calculer sa moyenne

```
#include <stdio.h>
#include <string.h>

int main(void)
{
double notes[5];
double somme = 0.0;
char mention[20];
int i;

for (i = 0; i < 5; i++) {
printf("Note %d : ", i + 1);
scanf("%lf", &notes[i]);
somme += notes[i];
}
if (somme / 5 >= 10) strcpy(mention, "Admis");
else strcpy(mention, "Ajourne");

printf("Moyenne : %.2f - %s\n", somme / 5, mention);
printf("La mention compte %d lettres\n", (int)strlen(mention));
return 0;
}
```

À retenir

- notes[5] n'existe pas dans un tableau int notes[5] : les indices vont de 0 à 4.
- Le '\0' final fait partie de la chaîne : prévoir toujours une case de plus que la longueur utile.
- Jamais de = ni de == pour les chaînes : strcpy pour copier, strcmp pour comparer.

Exercice

Écris un programme qui déclare une chaîne de 30 caractères, lit un prénom avec scanf, puis affiche sa longueur (sans utiliser strlen — compte les caractères toi-même jusqu'au '\0'). Explique pourquoi la boucle de comptage ne doit pas dépasser 30.

✔ Corrigé
int n = 0; while (prenom[n] != '\0') n++; puis afficher n. La boucle compare chaque case au caractère de fin ; elle s'arrête dès qu'elle le rencontre. La borne de 30 est la taille du tableau : si la chaîne a été lue proprement, le '\0' se trouve avant la fin ; la limite sert de ceinture de sécurité contre une chaîne mal terminée — sans \0, la boucle lirait au-delà du tableau, comportement indéfini. D'où la règle : toute chaîne manipulée doit toujours se terminer par '\0'.

Leçon 7 — Structures et types composés

Les tableaux regroupent des valeurs du même type ; mais une fiche d'étudiant mélange un nom (chaîne), un âge (entier) et une moyenne (réel). La structure (struct) crée un nouveau type composé qui rassemble ces champs sous un seul nom : struct Etudiant { char nom[30]; int age; double moyenne; };. Une fois le type défini, on déclare des variables de ce type comme n'importe quelles autres : struct Etudiant e1;

et l'on accède à chaque champ par le point : e1.age = 20; printf("%s", e1.nom);.

Les structures se combinent avec tout le reste du langage : tableaux de structures (un tableau de 30 étudiants pour une classe), structures passées à des fonctions (par copie, comme les types simples), fonctions qui remplissent ou affichent une fiche. C'est la première étape vers la programmation orientée objet des langages modernes : regrouper les données qui décrivent une même entité, au lieu de les éparpiller en tableaux parallèles difficiles à maintenir.

Le mot-clé typedef donne un surnom à un type, pour éviter de répéter struct partout : typedef struct { ... } Etudiant; permet ensuite d'écrire simplement Etudiant e1;. Un exemple complet vaut mieux qu'un discours : la fiche d'étudiant ci-dessous est saisie au clavier puis affichée proprement — c'est typiquement le genre de mini-programme demandé en contrôle de fin de module.

L'essentiel
struct crée un type composé de champs de types différents ; accès aux champs par le point (e1.nom). typedef donne un alias lisible. Tableaux de structures = gérer des collections d'entités complètes.

Exemple — Fiche étudiant complète

```
#include <stdio.h>

typedef struct {
char nom[30];
int age;
double moyenne;
} Etudiant;

int main(void)
{
Etudiant e;

printf("Nom : "); scanf("%29s", e.nom);
printf("Age : "); scanf("%d", &e.age);
printf("Moyenne : "); scanf("%lf", &e.moyenne);

printf("--- Fiche ---\n");
printf("Nom : %s\nAge : %d ans\nMoyenne : %.2f\n",
e.nom, e.age, e.moyenne);
return 0;
}
```

À retenir

- Un champ se lit et s'écrit avec le point : variable.champ.
- Chaque champ garde les règles de son type : scanf("%d", &e.age) exige le & comme pour un int ordinaire.
- Les structures préparent la POO : données d'une même entité regroupées au même endroit.

Exercice

Définis une structure Produit (nom, prix unitaire, quantité en stock), puis écris un programme qui saisit un produit et affiche la valeur totale de son stock (prix × quantité).

✔ Corrigé
typedef struct { char nom[30]; double prix; int quantite; } Produit; — dans main : Produit p; lire p.nom (%29s), p.prix (%lf, &), p.quantite (%d, &) ; total = p.prix * p.quantite ; afficher le total avec %.2f. Note que la lecture du nom ne prend pas de & (c'est un tableau, donc déjà une adresse), tandis que prix et quantite en exigent un : la règle du & se décide champ par champ, selon le type du champ.

Travaux dirigés (TD)

TD 1 — Calculatrice à menu

Écris un programme qui affiche un menu (1. Addition, 2. Soustraction, 3. Multiplication, 4. Division, 0. Quitter), lit le choix de l'utilisateur et les deux nombres, effectue l'opération choisie à l'aide d'un switch, et recommence jusqu'au choix 0 (boucle do-while). La division par zéro doit afficher un message d'erreur au lieu de calculer. Chaque opération sera réalisée par une fonction (additionner, soustraire, multiplier, diviser) prenant deux double et retournant un double.

✔ Corrigé du TD 1
Structure : quatre fonctions simples (diviser teste le diviseur et l'appelant gère le message, ou diviser retourne un code — la version simple teste dans le switch). Boucle do { afficher le menu ; scanf choix ; if choix entre 1 et 4 : lire a et b ; switch (choix) { case 1 : r = additionner(a,b); break; ... case 4 : if (b == 0) message d'erreur sinon r = diviser(a,b); } afficher r si pas d'erreur } while (choix != 0). Les break du switch sont indispensables : sans break, l'exécution « tombe » dans le cas suivant et additionnerait puis soustrairait à la suite. Le do-while garantit l'affichage du menu au moins une fois.

TD 2 — Statistiques d'une classe

Un enseignant veut traiter les notes de ses 30 étudiants, stockées dans un tableau après saisie. Écris les fonctions : double moyenneClasse(double t[], int n), double noteMax(double t[], int n), int nombreAdmis(double t[], int n) (notes ≥ 10). Le programme principal saisit les notes (avec validation : on redemande tant que la note n'est pas entre 0 et 20), puis affiche les trois résultats.

✔ Corrigé du TD 2
moyenneClasse : somme dans une boucle, retour somme / n. noteMax : max = t[0] puis comparaison à chaque case, retour du max. nombreAdmis : compteur incrémenté quand t[i] >= 10. Validation dans la saisie : do { scanf } while (note < 0 || note > 20). Point de vigilance : les fonctions reçoivent le tableau ET sa taille n — un tableau passé en paramètre ne connaît pas sa propre longueur, et utiliser 30 en dur dans les fonctions les rendrait inutilisables pour une autre classe.

Travaux pratiques (TP)

TP 1 — Du pseudo-code au C : le bulletin de notes

Reprends le TD « Bulletin de notes complet » du cours INF-L1-01 (moyenne pondérée de 5 matières). 1) Écris le programme C complet : tableaux notes[5] et coefs[5] remplis par scanf, fonction moyennePonderee(notes, coefs, 5), affichage de la moyenne et de la décision. 2) Compile-le, corrige chaque erreur de compilation en notant son type (oubli de ; , de &, de ==...). 3) Teste avec les données du TD (14/3, 9/2, 12/1, 11/2, 8/2) et vérifie que tu retrouves 11,0 et « Admis ».

✔ Attendus du TP 1
Résultats attendus : moyenne pondérée 110/10 = 11,00, décision « Admis ». Les erreurs de compilation les plus probables et leur lecture : « expected ';' before… » = point-virgule oublié à la ligne précédente ; plantage à l'exécution sur une saisie = & oublié dans un scanf ; moyenne fausse de type 11 affiché 11 alors qu'un cas décimal échoue = division entière (diviser par (double)sommeCoefs ou déclarer l'accumulateur en double). Le journal des erreurs tenu pendant le TP est un livrable en soi : relire ses erreurs classiques avant un contrôle évite de les refaire.

Exercices corrigés supplémentaires

Exercice 1

Écris un programme qui lit un entier et affiche s'il est positif, négatif ou nul.

✔ Corrigé
if (n > 0) « Positif » else if (n < 0) « Négatif » else « Nul ». La cascade teste les cas dans n'importe quel ordre tant qu'ils s'excluent ; le else final attrape le seul cas restant (0).

Exercice 2

Écris une fonction int estPair(int n) qui retourne 1 si n est pair, 0 sinon, et un programme qui compte combien de nombres pairs l'utilisateur saisit (il saisit 10 nombres).

✔ Corrigé
int estPair(int n) { return n % 2 == 0; } — l'expression de comparaison vaut déjà 1 ou 0 en C. Le programme : boucle de 10 lectures, compteur incrémenté quand estPair(n) est vrai. Astuce de style : une fonction dont le nom commence par « est » doit retourner un booléen — les noms parlants sont une documentation gratuite.

Exercice 3

Que contient la chaîne mot après : char mot[6] = "IAM"; strcat(mot, "-ML"); ? Donne le contenu case par case et la valeur de strlen(mot).

✔ Corrigé
"IAM" occupe I, A, M, \0 (4 cases). strcat recopie « -ML » à la place du \0 : -, M, L, puis un nouveau \0 → la chaîne devient I, A, M, -, M, L, \0, soit 7 cases — mais le tableau n'en a que 6 ! C'est un débordement : strcat ne vérifie jamais la place disponible. strlen vaudrait 6 si tout se passait bien, mais le programme a déjà corrompu la mémoire. Règle : avant de concaténer, vérifier que strlen(a) + strlen(b) < taille du tableau de destination.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Le rôle du compilateur est de…

   A) Exécuter le programme ligne par ligne

   B) Traduire le code source en langage machine et signaler les erreurs de syntaxe

   C) Corriger les erreurs de logique

2. En C, 7 / 2 vaut…

   A) 3.5

   B) 3

   C) Erreur

3. scanf("%d", &age) : le symbole & sert à…

   A) Multiplier age

   B) Passer l'adresse de age pour y ranger la valeur lue

   C) Afficher age

4. if (x = 5)…

   A) Compare x à 5

   B) Affecte 5 à x puis teste une valeur toujours vraie

   C) Est une erreur de compilation

5. Dans un tableau int t[10], le dernier indice valide est…

   A) 10

   B) 9

   C) 0

6. Une chaîne C se termine toujours par…

   A) Un point-virgule

   B) Le caractère '\0'

   C) Un espace

7. Pour comparer deux chaînes a et b, on écrit…

   A) a == b

   B) a = b

   C) strcmp(a, b) == 0

8. Une variable locale à une fonction…

A) Est visible dans tout le programme

B) Naît à l'appel et disparaît au retour de la fonction

C) Doit être déclarée avec #define

9. Le passage de paramètres en C (types simples) se fait…

A) Par adresse : la fonction modifie l'original

B) Par valeur : la fonction travaille sur une copie

C) Sans copie ni adresse, par magie

10. L'accès au champ age d'une structure e s'écrit…

A) e->age

B) e[age]

C) e.age

Corrigés du QCM

1. Réponse B — La compilation traduit et vérifie la syntaxe ; les erreurs de logique restent la responsabilité du programmeur.

2. Réponse B — Division entre deux entiers = division entière : le résultat est 3.

3. Réponse B — scanf écrit à une adresse : sans &, comportement indéfini et plantage fréquent.

4. Réponse B — Un seul = affecte ; la comparaison s'écrit ==. Ce piège compile sans protester.

5. Réponse B — Les indices vont de 0 à taille−1, donc 9. t[10] est un débordement.

6. Réponse B — Le caractère nul marque la fin ; sans lui, les fonctions de chaînes lisent au-delà.

7. Réponse C — strcmp retourne 0 quand les contenus sont égaux ; == comparerait les adresses.

8. Réponse B — Portée locale : création à l'entrée, destruction à la sortie de la fonction.

9. Réponse B — Par valeur : l'original de l'appelant est protégé ; les pointeurs changeront la donne (cours suivant).

10. Réponse C — L'opérateur point donne accès aux champs d'une structure variable (la flèche servira pour les pointeurs de structures).

Résumé — les notions importantes

• Un programme C s'écrit dans un fichier .c, se compile (GCC) puis s'exécute ; main est le point d'entrée, <stdio.h> fournit printf et scanf.

• Types de base : int, float, double, char ; initialiser chaque variable ; la division entre entiers est entière, % donne le reste.

• printf formate avec %d, %f/%.2f, %c, %s ; scanf range les saisies à des adresses (&variable), sauf les chaînes lues par leur nom.

• == compare et = affecte ; for/while/do-while répètent, break et continue ajustent les boucles, switch aiguille sur des valeurs.

• Les fonctions découpent le programme ; prototype, définition, appel ; passage par valeur, variables locales à privilégier.

• Tableaux indexés de 0 à taille−1 ; chaînes = tableaux de char terminés par '\0' ; strcpy, strcmp, strlen de <string.h>.

• struct compose des données hétérogènes en un type nouveau (la fiche étudiant), typedef le rend lisible : c'est la base des programmes de gestion.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Installation de l'environnement et premier programme en C
Un programme C est un fichier texte compilé en langage machine avant d'être exécuté. ; Les erreurs de compilation (syntaxe) se distinguent des erreurs de logique (résultat faux). ; Chaque instruction se termine par un point-virgule ; les blocs sont délimités par des accolades { }.

Fiche 2 — Variables, constantes et opérateurs
int = entiers, float/double = réels, char = un caractère ; double est le choix par défaut pour les réels. ; 7 / 2 = 3 (division entière) mais 7.0 / 2 = 3.5 : le type des opérandes décide du résultat. ; Une variable non initialisée contient une valeur imprévisible : toujours initialiser.

Fiche 3 — Entrées et sorties : printf et scanf
Le format doit correspondre au type : %d ↔ int, %lf ↔ double dans scanf, %f suffit dans printf. ; Le & devant la variable dans scanf n'est pas décoratif : c'est l'adresse où ranger la valeur. ; Toujours guider l'utilisateur par un message avant chaque saisie.

Fiche 4 — Conditions et boucles en C
if (x = 5) affecte au lieu de comparer : toujours écrire == dans un test. ; La borne d'une boucle se vérifie en simulant le premier et le dernier tour. ; do-while valide une saisie naturellement : on demande, puis on recommence si la valeur est hors limites.

Fiche 5 — Fonctions : déclaration, appel et portée des variables
Sans prototype ni définition avant l'appel, le compilateur refuse la fonction inconnue. ; Le paramètre est une copie : la fonction ne modifie pas la variable de l'appelant (types simples). ; Une variable locale n'existe que pendant l'exécution de sa fonction.

Fiche 6 — Tableaux et chaînes de caractères
notes[5] n'existe pas dans un tableau int notes[5] : les indices vont de 0 à 4. ; Le '\0' final fait partie de la chaîne : prévoir toujours une case de plus que la longueur utile. ; Jamais de = ni de == pour les chaînes : strcpy pour copier, strcmp pour comparer.

Fiche 7 — Structures et types composés
Un champ se lit et s'écrit avec le point : variable.champ. ; Chaque champ garde les règles de son type : scanf("%d", &e.age) exige le & comme pour un int ordinaire. ; Les structures préparent la POO : données d'une même entité regroupées au même endroit.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — APPRENDRE LE C [TUTO PROGRAMMATION COMPLET DÉBUTANT]

Chaîne : Comment Coder · Langue : Français · Niveau : Débutant
Le langage C de A à Z pour démarrer : installation de l'éditeur et du compilateur, premier programme, variables, scanf, conditions, boucles, fonctions, tableaux, chaînes, pointeurs et structures, avec des mini-projets. La vidéo-compagnon de tout le cours.

https://www.youtube.com/watch?v=q6FcVUFM42o

Leçon 3 — Partie 1 - Langage C – Bases fondamentales bibliothèques, printf scanf, mémoire et premier programme

Chaîne : Souleymane H Maizaboura · Langue : Français · Niveau : Débutant
Les bibliothèques standard (stdio.h…), printf en détail avec les séquences d'échappement, scanf et les spécificateurs de format (%d, %f, %c, %s), et la notion de mémoire — exactement la leçon 3, en cours filmé.

https://www.youtube.com/watch?v=A0XAV9JlZO8

Leçon 4 — Langage C - Partie 1 - Les Variables, Les Conditions, Les Boucles et Les Fonctions

Chaîne : KinSoft Lab · Langue : Français · Niveau : Débutant
Les notions de base du C avec l'IDE Code::Blocks : variables, conditions, boucles et fonctions démontrées en direct — parfait pour réviser les leçons 2, 4 et 5 avant un contrôle.

https://www.youtube.com/watch?v=M5RIPktv0KU

Leçon 6 — Langage C #2 Exercices complets (4K) scanf printf variables mémoire RAM Débutant à Avancé Souleymane

Chaîne : Souleymane H Maizaboura · Langue : Français · Niveau : Débutant à avancé
Des exercices complets et corrigés en vidéo : printf et scanf, variables int/float/double, division entière contre division réelle, opérations et modulo, avec l'explication de ce qui se passe dans la mémoire RAM.

https://www.youtube.com/watch?v=7G3KvWq_sCI

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Kernighan & Ritchie — « Le langage C » (l'ouvrage des créateurs du langage, référence historique et technique).

• Documentation et tutoriels officiels GCC (gcc.gnu.org) pour la compilation et les messages d'erreur.

• cppreference — référence du langage C (fr.cppreference.com) : fiches précises sur printf, scanf et les fonctions de <string.h>.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.